

Submission Requirements (PDF)

1. GitHub Link

Same repository: IT342-Lastname-AppName

<https://github.com/Andre-12-maker/IT342-Policios-CampusBites.git>

2. Final Commit

Message:

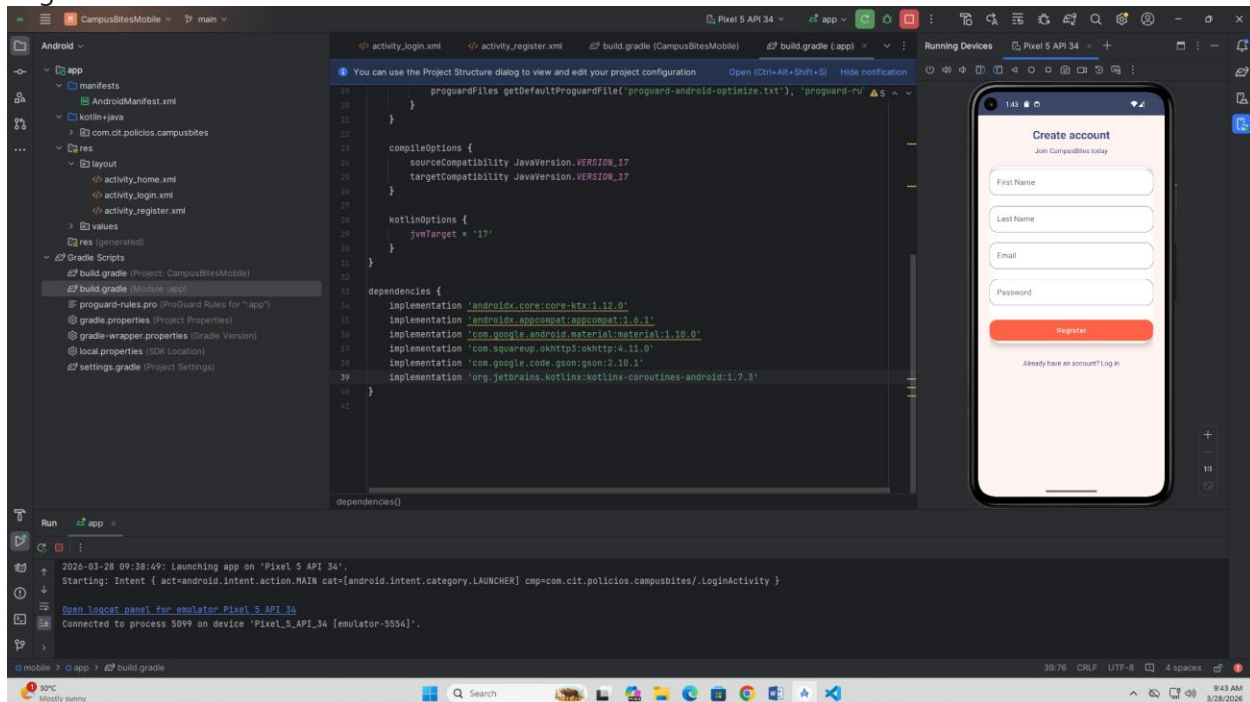
IT342 Phase 2 – Mobile Development Completed

Include commit link/hash

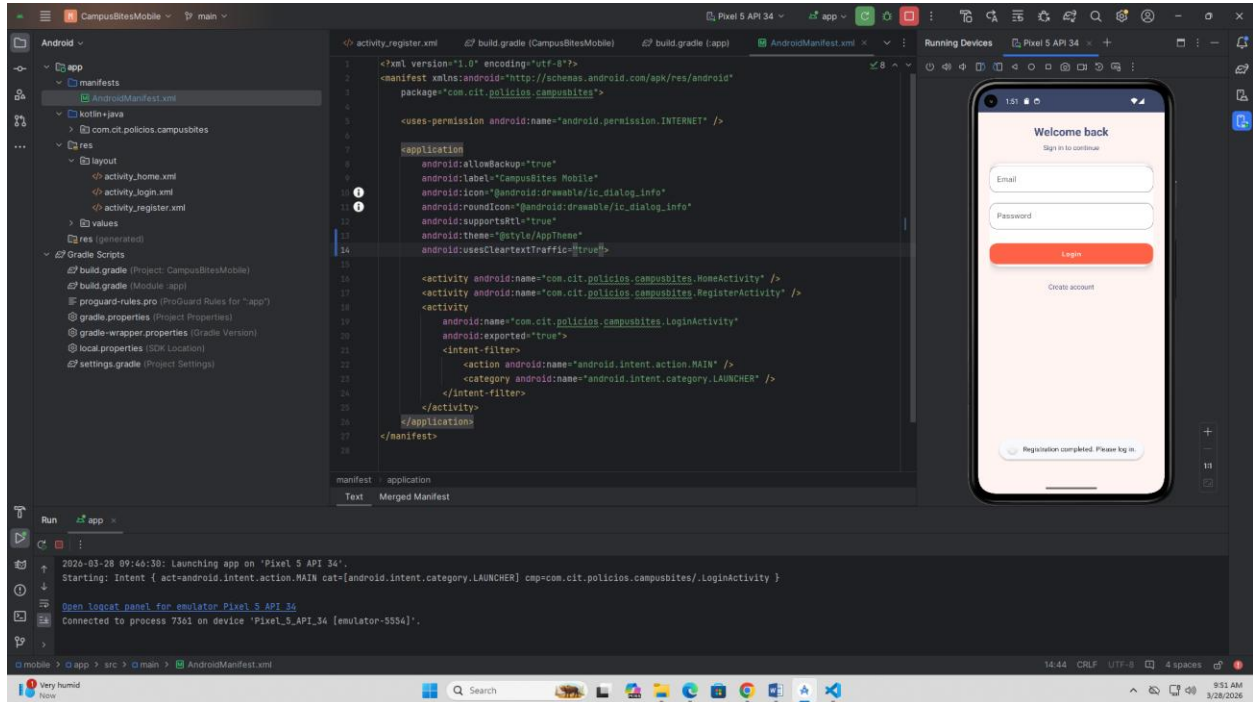
c9bf39b8930b86f99673e377209965351f2b9169

3. Screenshots

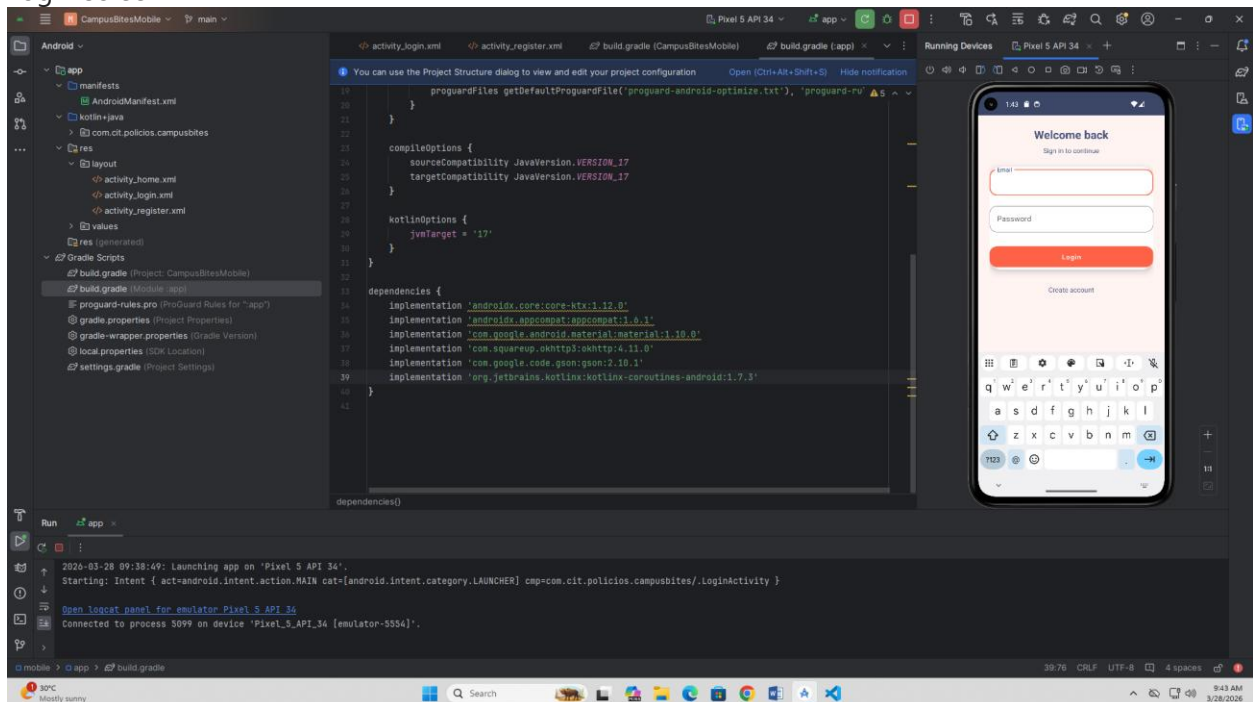
- Registration screen



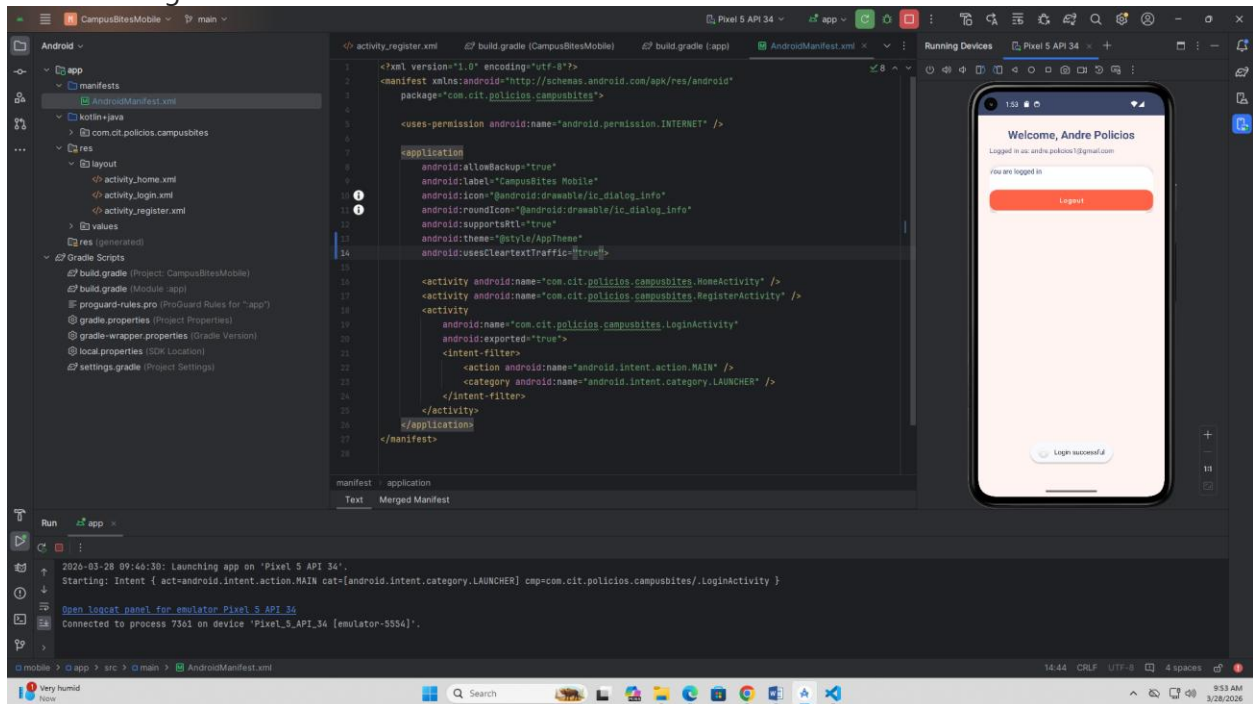
- Successful registration



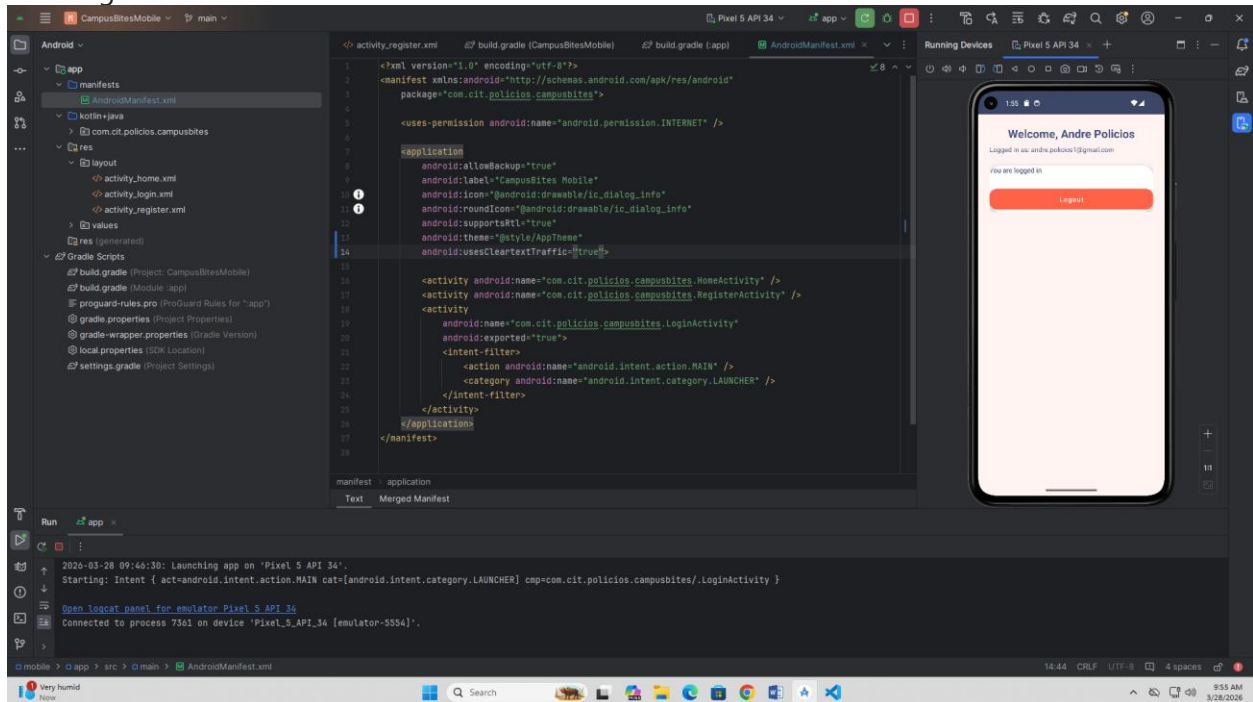
- Login screen



- Successful login



- After login screen



- Database record

The screenshot displays the MongoDB Cloud Portal interface. The browser address bar shows the URL: `cloud.mongodb.com/v2/69ab776bf137bec3a720da34/explorer/69ab78991db31f1704402bdc/CampusBitesDB/users/find`. The interface includes a left sidebar with 'Data Explorer' and 'CLUSTERS' sections. The main content area shows the 'users' collection in the 'CampusBitesDB' database. A search bar is present with a query filter. Below the search bar, there are buttons for 'ADD DATA', 'UPDATE', 'DELETE', and 'EXPORT CODE'. The document list shows three records. The first record is expanded, displaying its JSON structure:

```
{
  "_id": "69ab8536265d4fb703db77d6",
  "firstName": "Andre",
  "lastName": "Policios",
  "email": "test@cit.edu",
  "password": "52a510WvvdDp5NZLXL7qhystxp05H6UtoK35J249rsKyr1R3pl0ERMe035",
  "role": "CUSTOMER",
  "_class": "edu.cit.policios.campusbites.entity.User"
}
```

The second and third records are also visible but not expanded. The bottom of the screen shows the system status 'All Good' and the date '3/28/2026'.

4. Short Summary (1 Page)

How registration works

- The mobile app opens RegisterActivity.
- User enters first name, last name, email, and password.
- The app validates that none of the fields are empty.
- It creates a User object and runs ApiClient.register(user) on a background Thread.
- ApiClient.register serializes the user as JSON with Gson and sends it to POST <http://10.0.2.2:8080/auth/register>.
- If the backend returns success, the app shows a success toast and navigates to LoginActivity.
- If the call fails, the app shows an error toast with the API message.

How login works

- The mobile app opens LoginActivity.
- User enters email and password.
- The app validates both fields are filled.
- It runs ApiClient.login(email, password) on a background Thread.
- ApiClient.login sends JSON to POST <http://10.0.2.2:8080/auth/login>.
- On success, the backend returns a User object and the app starts HomeActivity.
- The app passes the user name and email into the home screen and displays a login success toast.
- On failure, the app shows a toast with the login error.

API integration used

- Networking is implemented in ApiClient.kt.
- Uses OkHttp client for HTTP requests.
- Uses Gson for JSON serialization/deserialization.
- Requests are sent to the local emulator host 10.0.2.2:8080.
- Two endpoints are used: /auth/register for registration
- /auth/login for login
- The mobile client expects the backend to return a User JSON object on success.
- Errors are converted into failed Result objects and displayed to the user.

Submission Format

File name:

IT342_Phase2_Mobile_ProjectName_Lastname.pdf